The Case Against std::execution For Networking

Document Number:	P4058R0
Date:	2026-03-13
Audience:	LEWG, SG1

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Five companion papers establish that `std::execution` cannot serve as the foundation for standard networking without losing data, bypassing its own composition algebra, or converting routine network events into exceptions.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

Disclosure

We developed and maintain Corosio[1] and Capy[2] and believe coroutine-native I/O is the correct foundation for networking in C++. We provide evidence, we state our case, and we defer respectfully to the committee.

The authors regard `std::execution` as an important contribution to C++ and support its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper or its companions argues for removing, delaying, or diminishing `std::execution`. The authors' position is narrower: that networking and stream I/O present a compound-result structure that the three-channel model was not designed to carry, and that this domain is better served by a coroutine-native facility that can coexist with senders and interoperate where the domains meet. Two models, each correct for its domain, is a stronger standard than one model asked to serve both.

Opening Statement

We present five exhibits. Taken individually, each documents a structural limitation of `std::execution` (P2300R10[3]) and `std::execution::task` (P3552R3[4]) when applied to networking. Taken together, they establish that the sender three-channel model is not a universal async abstraction - it is a powerful framework with a bounded domain, and networking lies outside that domain.

The question before the committee is not whether `std::execution` is good. It is whether `std::execution` can serve as the foundation for standard networking. We submit that the evidence answers this question.

Exhibit A establishes that operations partition into two classes based on postcondition structure, and that the three-channel model fits only one. Exhibit B constructs four sender-based echo servers from the C++26 specification and shows that no construction achieves both full data preservation and channel-based composition. Exhibit C identifies the abstraction floor - the boundary where compound I/O results must be reduced to binary outcomes before crossing into sender territory. Exhibit D demonstrates that a single class template consumes senders from coroutine-native code without `std::execution::task`, without exceptions for

`error_code`, and without `AS-EXCEPT-PTR`. Exhibit E documents that seven production coroutine libraries independently converged on a one-parameter task type, and that `std::execution::task` fragments on contact with itself.

The evidence is drawn entirely from the companion papers, from the `std::execution` specification, and from the published words of the framework's own authors and reviewers.

Exhibit A: The Error Model Partition

D4054R0[5], "Two Error Models."

Chris Kohlhoff identified the structural difficulty in P2430R0[6] (2021):

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

Dietmar Kühl reached the same conclusion independently in P2762R2[7] (2023):

> *"some of the error cases may have been partial successes...using the set_error channel taking just one argument is somewhat limiting"*

The reason is structural. Operations partition into two classes based on what they promise to return. Infrastructure operations - `malloc`, `fopen`, `pthread_create`, GPU kernel launch, timer arm - have binary outcomes: the postcondition was satisfied or it was not. Compound-result operations - `read`, `write`, `accept`, `from_chars` - return a status and associated data, always paired. The OS delivers both values together. POSIX has done so since 1988[8]. io_uring delivers `(res, flags)` in one CQE. IOCP delivers `(BOOL, lpNumberOfBytesTransferred, lpOverlapped)` in one call.

The sender three-channel model assigns semantic meaning to each channel: `set_value` for success, `set_error` for failure, `set_stopped` for cancellation. For infrastructure operations, the mapping is unambiguous. For compound-result operations, the mapping requires a choice that either loses data, bypasses the channels, or redefines what the channels mean.

The partition is not about asynchrony. It is about postcondition structure. `std::from_chars` is synchronous and returns a compound result. `read` may be synchronous or asynchronous and returns a compound result. The three-channel model faces the same structural difficulty in both cases.

Kohlhoff identified the problem in 2021. Kühl documented it in 2023. Kirk Shoop observed in P2471R1[9] (2021) that completion tokens translating to senders "must use a heuristic to type-match the first arg." To our knowledge, no published paper resolves the compound-result channel-routing problem. It remains open.

The three-channel model is correct for infrastructure operations. It cannot represent compound-result operations without loss.

Exhibit B: The Constructed Comparison

D4053R0[10], "Sender I/O: A Constructed Comparison."

Four sender-based TCP echo servers were constructed from the C++26 specification (P2300R10[3], P3552R3[4]) and compared against a coroutine-native echo server. The echo server is deliberately minimal. The compound-result problem is per-operation - adding protocol complexity adds more call sites with the same trade-off, not a different one.

"Just use `set_value`" (D4053R0[10] Section 5) routes both values through `set_value`. Both values visible. No exceptions. But `set_error` serves no purpose - `when_all` does not cancel siblings on I/O failure, `upon_error` is unreachable, `retry` does not fire. The three-channel model reduces to one channel.

"Just split the result" (D4053R0[10] Section 6) routes the error code through `set_error` and captures the byte count in shared state. All three channels in use. But the byte count bypasses the channels - `retry` fires on `set_error` but the byte count reflects the previous stage, not the failed one. Shared mutable state across continuation boundaries reintroduces the hazard the sender model was designed to eliminate.

"Just use `set_error`" (D4053R0[10] Section 7) routes errors through `set_error`. Channel-based composition works. But every `ECONNRESET` requires `make_exception_ptr` plus `rethrow_exception`. The byte count is destroyed. 500 of 1,000 bytes written before a connection reset - gone.

"Just decompose it" (D4053R0[10] Section 8) decomposes with `let_value`. The handler sees both values. Classification happens with full application context. But `set_error` takes a single argument. The byte count is destroyed when the error code crosses into the error channel. The decomposition point moved from the I/O layer to the application. The information loss did not.

No construction achieves both full data preservation and channel-based composition without shared state or exceptions. The invitation in D4053R0[10] Section 13 stands: construct a sender-based echo server that preserves compound I/O results, retains channel-based composition without altering the completion signatures

expected by generic sender algorithms from those algorithms' specified behavior, avoids exception round-trips, and uses only the algorithms and sender adapters specified in P2300R10[3], P3552R3[4], and P3149R9[24]. We will incorporate any such construction and re-evaluate every finding.

Four approaches. Four trade-offs. The coroutine-native approach pays none of them.

Exhibit C: The Abstraction Floor

D4056R0[11], "Producing Senders from Coroutine-Native Code."

Eric Niebler wrote in 2020[12]:

> *"That style of programming makes a different tradeoff, however: it is far harder to write and read than the equivalent coroutine. I think that 90% of all async code in the future should be coroutines simply for maintainability."*

The 90% and the 10% need a boundary. D4056R0[11] identifies that boundary: the abstraction floor.

Region	What the code sees
Above the floor	`error_code` alone - composition works
Below the floor	`(error_code, size_t)` - both values intact

Above the floor, outcomes are binary. The three channels map unambiguously. `when_all` cancels siblings on failure. `upon_error` is reachable. `retry` fires. The sender model works.

Below the floor, results are compound. The status code and the byte count arrive as a pair because they are a single result. The coroutine body inspects both values with full application context, performs protocol logic, and reduces the compound result to a binary outcome - an `error_code` - that crosses above the floor.

The bridge rejects compound I/O results at compile time at the sender boundary. An awaitable returning `(error_code, size_t)` cannot be wrapped as a sender directly. The constraint is structural, not nominal - it catches `io_result<size_t>`, `std::tuple<error_code, size_t>`, `std::pair<error_code, size_t>`, or any user-defined type with the same shape.

The coroutine body is the translation layer. The channels work above the floor. Below it, both values stay intact. The floor is not a limitation of the bridge - it is a recognition that the compound-result problem documented in Exhibits A and B has a structural solution: do not force compound results through three channels. Let the

coroutine handle them where both values are visible, and let the channels handle the binary outcome that emerges.

The abstraction floor is where the sender model's domain ends and the coroutine model's domain begins.

Exhibit D: The Consuming Bridge

D4055R0[13], "Consuming Senders from Coroutine-Native Code."

Jonathan Müller documented in P3801R0[14] (2025) that `std::execution::task` lacks symmetric transfer support, and that "a thorough fix is non-trivial and requires support for guaranteed tail calls." Dietmar Kühl confirmed the gap in P3796R1[15]. The sender-awaitable's `await_suspend` returns `void`, foreclosing symmetric transfer - the mechanism that prevents stack overflow in deep coroutine chains.

A single class template - `sender_awaitable` - consumes any `std::execution` sender from coroutine-native code. Operation state stored inline. `set_value`, `set_error`, `set_stopped` handled. Any sender pipeline - `when_all`, `then`, `let_value`, `on` - works. When the sender completes, the bridge posts the resumption back to the coroutine's originating executor. The coroutine resumes in the correct context regardless of where the sender executed.

The bridge inspects error completion signatures at compile time. If the sender advertises `set_error(std::error_code)`, the result is returned as a value - no exceptions. Genuine exceptions are rethrown. Static dispatch.

Property	`execution::task`	Bridge
Routine I/O errors become exceptions	Yes	No
Type erasure on connect	Yes	No
`AS-EXCEPT-PTR` for `error_code`	Yes	No
Zero allocations beyond frame	No	Yes

The bridge does not use `std::execution::task`. It does not require `std::execution::task`. The complete implementation is one class template, presented in D4055R0[13] Appendix A.

`std::execution::task` is not necessary to consume senders from coroutine-native code.

Exhibit E: Task Type Fragmentation

D4050R0[16], "On Task Type Diversity."

Gor Nishanov stated the design intent in P1362R0[17] (2018):

> *"The separation is based on the observation that coroutine types and awaitables could be developed independently of each other by different library vendors."*

And at CppNow 2017[18]:

> *"We did not want to tie it to a particular task library. [...] We wanted them to be open."*

Seven production coroutine libraries independently honored that intent. asyncpp, Boost.Cobalt, Capy, cppcoro, aiopp, libcoro, and folly::coro each define a one-parameter task type: `task<T>`. The interface converged. The machinery diverged - Cobalt embeds intrusive list nodes for cancellation, folly::coro integrates with Facebook's fiber scheduler, Capy propagates `io_env` through `await_suspend`. Each promise enforces domain-specific safety invariants. The diversity is not fragmentation. It is fitness.

P3552R3[4] adds a second template parameter: `Environment`. No default. The parameter exists because `task` is both a coroutine return type and a sender. Every function that returns `task` is coupled to the sender environment model.

Two libraries that independently define semantically identical empty environment structs produce incompatible task types. A library that upgrades from the default environment to a custom one changes its return type and breaks every downstream caller. The `Environment` parameter is an open query-response protocol with unbounded query sets - a type-erased `any_environment` that preserves the sender model's compile-time properties does not exist and cannot exist, because the query set is open-ended.

The only production two-parameter coroutine type - Asio's `awaitable<T, Executor>` - provides a type-erased default. Even so, users who replace `any_io_executor` with `io_context::executor_type` for performance immediately hit type incompatibility, documented across multiple StackOverflow questions[19][20][21]. Asio's case is mild because the escape hatch exists - users can fall back to `any_io_executor` at the cost of one virtual dispatch per operation. P3552R3[4]'s `Environment` has no such escape hatch.

One-parameter task types compose by construction. Two-parameter task types fragment by default. `std::execution::task` is neither necessary (Exhibit D) nor composable for networking.

Closing Argument

The five exhibits are independent findings. Each stands on its own evidence. Together, they form a single conclusion.

Exhibit A established that operations partition into two error models. The three-channel model is correct for one. Exhibit B demonstrated that no sender construction achieves both data preservation and channel-based composition for the other. Exhibit C identified the abstraction floor - the boundary where compound results must be reduced to binary outcomes before the channels can work. Exhibit D showed that senders are consumable from coroutine-native code without `std::execution::task`. Exhibit E showed that `std::execution::task` fragments where networking needs composition.

The pattern is consistent. At every point where `std::execution` meets networking - error handling, data preservation, task composition, bridge architecture - the framework requires the networking code to sacrifice something the coroutine-native approach preserves: a byte count, a non-throwing error path, a stable return type, or the composition algebra itself.

`std::execution` is not a failed design. It serves GPU dispatch, compile-time work graphs, thread pool submission, timer management, and every other infrastructure operation with binary outcomes. The reference implementation - stdexec[22] - targets exactly these domains. The three-channel model is correct for them. The finding is domain, not defect.

But when a framework claims universality, the burden of evidence is proportional to the claim. `std::execution` did not bring extraordinary evidence for networking. No sender-based networking library has demonstrated production-scale I/O with the three-channel model. No published paper resolves the compound-result channel-routing problem identified by Kohlhoff in 2021. No mechanism in the specification delivers a frame allocator to `operator new` without `allocator_arg_t`. No `any_environment` preserves the compile-time properties that justify the model's complexity.

The five exhibits suggest the opposite of universality. The framework is powerful within a bounded scope. Networking lies outside that scope.

We have presented the evidence. We submit that `std::execution` is not universal - that it serves the domains for which it was designed, and that networking is not among them. Ship `std::execution` for those domains. Pursue a coroutine-native design for networking.

Acknowledgments

The author thanks Steve Gerbino for co-developing the constructed comparison and bridge implementations; Mungo Gill for feedback and co-authorship on prior work; Chris Kohlhoff for identifying the partial-success problem in P2430R0[6]; Dietmar Kühl for the channel-routing enumeration in P2762R2[7] and for `beman::execution`; Eric Niebler, Michał Dominiak, and Lewis Baker for `std::execution`; Gor Nishanov for the coroutine model's explicit support for task type diversity; Peter Dimov for the refined channel mapping and for identifying the frame allocator propagation gap; Jonathan Müller for confirming the symmetric transfer gap in P3801R0[14]; Kirk Shoop for the completion-token heuristic analysis in P2471R1[9]; Fabio Fracassi for P3570R2[23]; Klemens Morgenstern for Boost.Cobalt and the cross-library bridges; Ian Petersen, Jessica Wong, and Kirk Shoop for `async_scope`; Maikel Nadolski for work on `execution::task`; Mark Hoemmen for insights on `std::linalg` and the layered abstraction model; Herb Sutter for identifying the need for constructed comparisons; Ville Voutilainen and Jens Maurer for reflector discussion; and Mohammad Nejati and Michael Vandeberg for feedback.

References

1. cppalliance/corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio
2. cppalliance/capy - Coroutine primitives library. https://github.com/cppalliance/capy
3. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
4. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
5. D4054R0 - "Two Error Models" (Vinnie Falco, 2026). https://wg21.link/d4054r0
6. P2430R0 - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). https://wg21.link/p2430r0
7. P2762R2 - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). https://wg21.link/p2762r2
8. IEEE Std 1003.1-2024 - POSIX `read()` / `write()` specification. https://pubs.opengroup.org/onlinepubs/9799919799/
9. P2471R1 - "NetTS, ASIO and Sender Library Design Comparison" (Kirk Shoop, 2021). https://wg21.link/p2471r1
10. D4053R0 - "Sender I/O: A Constructed Comparison" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4053r0
11. D4056R0 - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4056r0

12. Eric Niebler, ‘‘Structured Concurrency,’’ blog post, November 2020. https://ericniebler.com/2020/11/08/structured-concurrency/

13. D4055R0 - ‘‘Consuming Senders from Coroutine-Native Code’’ (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4055r0

14. P3801R0 - ‘‘Concerns about the design of std::execution::task’’ (Jonathan Müller, 2025). https://wg21.link/p3801r0

15. P3796R1 - ‘‘Coroutine Task Issues’’ (Dietmar Kühl, 2025). https://wg21.link/p3796r1

16. D4050R0 - ‘‘On Task Type Diversity’’ (Vinnie Falco, 2026). https://wg21.link/d4050r0

17. P1362R0 - ‘‘Incremental Approach: Coroutine TS + Core Coroutines’’ (Gor Nishanov, 2018). https://wg21.link/p1362r0

18. Gor Nishanov, ‘‘C++17 coroutines for app and library developers,’’ CppNow 2017. https://www.youtube.com/watch?v=proxLbvHGEQ

19. Boost asio using concrete executor type with c++20 coroutines causes compilation errors - StackOverflow (2024). https://stackoverflow.com/questions/79115751

20. Can I co_await an operation executed by one io_context in a coroutine executed by another in Asio? - StackOverflow (2022). https://stackoverflow.com/questions/73517163

21. How to create custom awaitable functions that can be called with co_await inside asio::awaitable functions - GitHub asio#795 (2022). https://github.com/chriskohlhoff/asio/issues/795

22. stdexec - Reference implementation of std::execution. https://github.com/NVIDIA/stdexec

23. P3570R2 - ‘‘Optional variants in sender/receiver’’ (Fabio Fracassi, 2025). https://wg21.link/p3570r2

24. P3149R9 - ‘‘`async_scope` - Creating scopes for non-sequential concurrency’’ (Ian Petersen, Jessica Wong, Kirk Shoop, et al., 2025). https://wg21.link/p3149r9